🗡️ SLiCER v2 — System Architecture & User Manual

Safe Long-term Cloud Event Archival at the Edge

Version 2.0 Technical Reference & Operational Guide | LASIGE / FCUL & UTFPR Campus Pato Branco

1. Executive Overview & Evolution

SLiCER (Safe Long-term Cloud Event Archival) is a decentralized SIEM (Security Information and Event Management) and Threat Hunting engine engineered to operate at the network edge. It ingests, parses, indexes, compresses, encrypts, and synchronizes thousands of security events per second (EPS) across a resilient multi-tier storage hierarchy combining local disks, mapped single/multi-cloud sync folders (such as Google Drive, Dropbox, Vawlt.io, or the academic multi-cloud file system *Charon*), and cold object storage (AWS S3).

Currently in **Version 2 (v2)**, SLiCER introduces major architectural enhancements over the original Version 1 prototype (published at IEEE PRDC 2019, now deprecated). Version 2 eliminates concurrency bottlenecks via an *Atomic State Push* pattern, supports multi-format event ingestion (CEF and JSON), introduces hybrid Lucene analyzer wrappers for exact punctuation preservation, provides real-time NDJSON streaming to an interactive Web Dashboard, and implements surgical forensic block exports with integrated FinOps S3 egress cost estimation.

2. Core Architectural Pillars

- **High-Throughput Concurrent Ingestion (+5,300 EPS):** Asynchronous TCP socket listeners (`TcpEventSource`) and directory spool monitors (`DirectoryMonitorSource`) route events into lock-free concurrent queues (`ConcurrentLinkedQueue`).
- **Atomic State Push & Race-Condition Immunity:** To prevent micro-block fragmentation during disk I/O flushes (~370ms compression/indexing windows), byte size counters and epoch boundaries are updated atomically inside the synchronized disk-flush critical section rather than during raw event arrival.
- **Multi-Format Parsing Engine:** Extensible `EventParser` interface with built-in `CefParser` (ArcSight Common Event Format) and `JsonParser` (Jackson-based key-value extraction), dynamically routed per device by `EventRouter`.
- **Hybrid Indexing & Dynamic Bloom Filters:** Each block generates an Apache Lucene inverted index alongside a probabilistic Bloom Filter (`.bf`). Bloom filter capacity self-tunes dynamically using a 5-block moving average of unique inserted tokens, providing O(1) block skipping for simple equality queries.
- **Per-Field Analyzer Wrapper:** Lucene indexing utilizes a `PerFieldAnalyzerWrapper` that applies `StandardAnalyzer` to free-text fields while enforcing `KeywordAnalyzer` on structured identifiers (IP addresses, device names with dots/hyphens like `ids.snort` or `Windows-AD`).
- **Client-Side Envelope Encryption (AES-256-GCM):** Prior to leaving the hot cache, data and index archives (`.tar.xz`) are encrypted locally using AES-256 in Galois/Counter Mode (GCM) with a `SLCR` magic header (and legacy fallback compatibility). The 256-bit key is derived in-memory from a Master Password entered at daemon startup; no cloud KMS is required.
- **Cryptographic Hash Chaining & Central Ledger:** Every block stores a `chain.meta` file linking to the `PREVIOUS_BLOCK_HASH` (starting from `GENESIS_BLOCK`). Upon synchronization, the SHA-256 digest of the compressed archive is permanently recorded in monthly JSON manifests (`manifest_YYYY-MM.json`).
- **Zero-Loss Graceful Shutdown:** A JVM Shutdown Hook intercepts termination signals (`SIGTERM` / IDE Stop), flushes all open memory buffers across all device managers, packages residual blocks, and invokes `CloudSyncWorker.runSyncJobNow()` synchronously before process exit.

3. Multi-Tier Storage Hierarchy & Data Lifecycle

SLiCER strictly segregates storage directories by operational tier. Understanding this topology is essential for system administration and forensic workflows.

```
/storage
  ├── /ingest_cache           # Tier 1: Hot Tier (Local Buffer, Active & Recent Indexes)
  │    └── /<device_name>
  │         ├── /data         # Active raw log buffer (newblockRaw.slc) & closed .tar.xz
  │         ├── /index        # Active Lucene dir (newblock) & closed .tar.xz / .bf
  │         └── chain_state.txt # Last SHA-256 block hash for local blockchain continuity
  ├── /tmp_charon             # Tier 2: Warm Tier (Mapped Multi-Cloud / DropDir)
  │    └── /<device_name>/<block_name>/*.enc
  ├── /exports                # Tier 3: Investigation Tier (Decrypted Forensic Archives)
  │    └── /<device_name>/<block_name>.tar.xz
  └── /mock_data              # Optional backup directory for ThreatSimulator (.gz)
/ledgers
  └── /<device_name>          # Tier 4: Immutable Audit Tier (Central Manifests)
       └── manifest_YYYY-MM.json
```

Tier / Directory	Role	Lifecycle & Pruning Behavior
Hot Tier `/ingest_cache`	Real-time buffering & local indexing	Events append to `newblockRaw.slc`. When `maxSizeMb` or `maxIntervalMinutes` is reached, the block closes into `.tar.xz`. Once synced to the cloud, **Egress-Drop** immediately deletes the raw `data/*.tar.xz` file to save disk space. Local indexes remain until total device cache exceeds `maxIngestCacheMb`, triggering oldest-first pruning of blocks marked with `.synced`.
Warm Tier `/tmp_charon`	Edge / Mapped Cloud DropDir	Holds AES-256 encrypted envelopes (`.tar.xz.enc` and `.bf.enc`). Can be pointed to any local path mounted by Google Drive, Dropbox, OneDrive, Vawlt.io, or Charon. Queried automatically when a block is missing from the Hot Tier, avoiding AWS S3 egress fees.
Cold Tier `AWS S3 Bucket`	Long-term archival	Stores encrypted blocks under the S3 key structure `<device>/<blockName>/<file>.enc`. Accessed on-demand only when blocks are absent from both Hot and Warm tiers.
Immutable Tier `/ledgers`	Historical registry	Stores `manifest_YYYY-MM.json` mapping every closed `blockName` to its SHA-256 hash. Never pruned. Enables the query and export engines to discover historical blocks residing exclusively in Cold Storage.
Forensic Tier `/exports`	Decrypted output	Destination for surgical forensic extractions triggered from the Web Dashboard. Contains decrypted, standard `.tar.xz` archives ready for external audit.

4. Configuration & Customization Guide

To adapt SLiCER to different network environments, storage backends, or log schemas, administrators must configure specific files before or during deployment.

4.1 Main Configuration File (`application.yml` / `AppConfig`)

The YAML configuration file (loaded via `AppConfig.load()`) governs network ports, block rotation thresholds, field indexing rules, storage paths, and cloud credentials. Below is the complete reference structure:

```yaml
slicer:
  ingest:
    tcpPort: 5140                          # TCP port for incoming CEF/JSON log streams
    spoolDir: "storage/spool"              # Directory monitored for batch log files
  api:
    port: 7000                             # Web Dashboard and REST API port (Javalin)
  block:
    maxSizeMb: 5                           # Rotate block when raw logs reach N Megabytes
    maxIntervalMinutes: 15                 # Rotate block every N minutes regardless of size
    bufferLimit: 1000                      # In-memory event queue threshold before disk flush
    defaultFieldsToIndex:                  # Default fields indexed in Lucene & Bloom Filter
      - "src"
      - "dst"
      - "act"
      - "suser"
      - "dvc"
      - "status"
    deviceFields:                          # Per-device custom indexing overrides
      firewall.ciscoasa:
        - "src"
        - "dst"
        - "act"
        - "proto"
      sgbd.mysql:
        - "suser"
        - "src_ip"
        - "query"
        - "db"
      webserver.php:
        - "all"                            # Use "all" to index every parsed key-value pair
  storage:
    ingestCacheDir: "storage/ingest_cache"  # Hot Tier path
    cloudDropDir: "storage/tmp_charon"      # Warm Tier path (leave blank "" to disable)
    maxIngestCacheMb: 5120                  # Max local cache per device (e.g., 5GB) before pruning
    syncIntervalSeconds: 30                 # CloudSyncWorker execution interval
  aws:
    bucketName: "my-slicer-archival-bucket" # S3 Bucket name (leave blank "" to disable S3)
```

> 💡 **How to Select Which Fields Are Indexed (Storage vs. Search Granularity)**
>
> By default, SLiCER always preserves the complete, unindexed raw log inside the compressed `.slc` data archive. However, to keep Lucene indexes and Bloom filters compact on edge devices, you can restrict which fields are indexed via `defaultFieldsToIndex` and `deviceFields` in `application.yml`:
>
> - **Specific Fields:** List exact field keys (e.g., `src`, `dst`, `suser`, `query`). Only these keys will be searchable via field-specific queries.
> - **Full Indexing:** Set the list to `["all"]` (or leave both settings empty, which triggers the automatic fallback to `"all"` in `DeviceBlockManager`).

4.2 Adding or Modifying Devices in the Web Dashboard (`web/index.html`)

Because SLiCER partitions directories by `deviceId` (extracted from the `dvchost` or `dvc` field of incoming logs), the Web Dashboard sends the list of selected devices in the query payload. When onboarding new log sources, add a corresponding checkbox inside the `<div id="checkboxes">` container in `web/index.html`:

```
<!-- Inside web/index.html -->
<div id="checkboxes">
    <label for="all_dev"><input type="checkbox" id="all_dev" onchange="toggleAllDevices(this)" checked/> [T
    <label><input type="checkbox" class="dev-cb" value="my.new.device" onchange="updateMultiSelectText()" c
</div>
```

4.3 Configuring the Traffic Generator (`simulator.json`)

To test custom ingestion rates or device profiles, edit `simulator.json` in the project root directory:

```
{
  "targetHost": "127.0.0.1",
  "targetTcpPort": 5140,
  "saveToDisk": true,
  "diskOutputPath": "storage/mock_data",
  "devices": [
    { "name": "webserver.php", "eps": 719, "format": "JSON" },
    { "name": "sgbd.postgresql", "eps": 749, "format": "CEF" },
    { "name": "sgbd.mysql", "eps": 856, "format": "JSON" },
    { "name": "proxy.squid", "eps": 756, "format": "CEF" },
    { "name": "ids.snort", "eps": 669, "format": "CEF" },
    { "name": "firewall.sonicwall", "eps": 755, "format": "CEF" },
    { "name": "firewall.ciscoasa", "eps": 833, "format": "CEF" }
  ]
}
```

4.4 Extending Log Formats (Custom Parsers)

SLiCER natively parses **CEF** (headers + key=value extensions) and flat **JSON** objects. To add a new format (e.g., Syslog RFC 5424 or CSV):

1. Create a class implementing `br.edu.utfpr.slicer.parser.EventParser` (implementing `canParse(String)` and `parse(String)` returning a `ParsedEvent`).
2. Register the new parser instance in `SlicerDaemon.java` inside `List.of(new CefParser(), new JsonParser(), new MyCustomParser())`.

5. Installation, Compilation & Execution

5.1 Prerequisites & Build

- **Java Development Kit (JDK):** Version 17 or higher.
- **Apache Maven:** Version 3.8+.
- **AWS Credentials (Optional):** If S3 synchronization is enabled, configure standard AWS SDK credentials (via `~/.aws/credentials` or environment variables `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`). Region defaults to `SA_EAST_1` in `CloudSyncWorker`.

```
git clone https://github.com/username/slicer.git
cd slicer
mvn clean install
```

> ⚠️ **Eclipse IDE Troubleshooting (Package Declaration Mismatch)**
>
> If Eclipse displays the error *"The declared package br.edu.utfpr.slicer does not match the expected package src.main.java.br.edu.utfpr.slicer"*, right-click the project root in Project Explorer → **Maven** → **Update Project... (Alt+F5)** and click OK. This restores `src/main/java` as the proper source root.

5.2 Starting the SLiCER Daemon

Run the main class `br.edu.utfpr.slicer.SlicerDaemon`. Immediately check the console output:

```
================================================
   SLiCER Edge Node - Inicialização Segura
================================================
⚠️ IDE Mode - Senha Mestra:
```

Type your **Master Password** and press Enter. The daemon derives a 256-bit AES key in RAM, initializes the TCP ingestion server, starts the directory monitor, launches the Javalin REST/Web server, and schedules the `CloudSyncWorker`.

Crucial Security Note: Always use the exact same Master Password across sessions when reading existing blocks. Entering a different password generates a different AES key, causing GCM authentication tag failures when decrypting historical blocks from Charon or S3.

6. Operating the Threat Hunting Dashboard

Open a web browser and navigate to `http://localhost:7000` (or your configured API port).

Important: Never open `web/index.html` directly via double-click (`file:///` protocol), as modern browsers block relative `fetch('/api/query')` requests due to CORS policies, resulting in a `NetworkError`.

6.1 Simplified vs. Advanced Query Modes

- **Simplified Mode:** Enter comma-separated key-value pairs (e.g., `dst=10.0.0.177, act=drop`) or a single free-text term. Queries eligible for simple equality checks leverage the **Bloom Filter** to skip non-matching blocks in O(1) time without opening Lucene indexes.
- **Advanced Mode (Lucene Syntax):** Click `[ Alternar para Modo Avançado ]` to write full Boolean expressions, wildcards, and phrase queries. Note that the UI automatically normalizes `=` to `:`. Complex boolean queries bypass the Bloom Filter and evaluate directly against the block Lucene indexes.

Example Multi-Device Coordinated Attack Query:

```
dvc:ids.snort OR (dvc:webserver.php AND status:404) OR (dvc:sgbd.mysql AND query:"DROP TABLE logs") OR (dvc
```

6.2 NDJSON Streaming, Tabs & Tree-View

When you click **Pesquisar Logs**, the backend streams results line-by-line using Newline Delimited JSON (`application/x-ndjson`). The dashboard dynamically creates a dedicated tab for each matching device and groups events inside collapsible `<details>` block trees showing the exact event count per block.

6.3 FinOps "S3 Cost" Indicator

At the end of a search stream, the backend emits a `summary` packet containing `s3BytesDownloaded`. The dashboard calculates the estimated AWS S3 data transfer cost and updates the top-right badge:

- `☁️ Custo S3: Zero` **(Green):** All searched indexes and blocks were satisfied from the local Hot Cache or Warm Tier (Charon/Mapped Cloud Drive).
- `☁️ Custo S3: ~$0.000142` **(Amber):** Indicates the exact egress cost incurred by fetching cold blocks from AWS S3.

6.4 Surgical Forensic Block Export

Analysts can extract raw, decrypted `.tar.xz` blocks into `storage/exports/<device>/` in two ways:

1. **Single Block Export:** Click `⬇️ Decifrar e Exportar Bloco` next to any specific block in the tree view (calls `POST /api/export`).
2. **Selective Query-Context Export:** Click `⬇️ Baixar Todos os Blocos Listados Deste Device` at the top of a device tab. Instead of blindly downloading the entire historical ledger of that device, the frontend tracks only the blocks rendered in the current query view (`blocksInView[device]`) and sends that exact list to `POST /api/export/selected`.

7. Testing & Tier Simulation Workflows

You can empirically validate SLiCER's multi-tier fallback logic (Hot → Warm → Cold) by generating traffic with `ThreatSimulator`, stopping the simulator, and selectively deleting local files.

Rule of Thumb: Never delete files inside `/ledgers`! The ledger manifests are how SLiCER remembers which historical blocks exist in the cloud after local files are wiped.

1. **Test 1 — Hot Tier (Local Cache):** Run a query immediately after generating blocks. Blocks are read from `storage/ingest_cache/<device>`. Response is instant; **S3 Cost is Zero**.
2. **Test 2 — Warm Tier (Charon / Mapped Cloud Folder):** Wait for `CloudSyncWorker` to sync (`.synced` marker appears), then manually delete the block folders inside `storage/ingest_cache/<device>/index` and `/data`. Run the query or click Export. SLiCER reads the Ledger, misses the local cache, fetches the `.enc` files from `storage/tmp_charon/<device>`, and decrypts them in RAM. **S3 Cost remains Zero**.
3. **Test 3 — Cold Tier (AWS S3):** Delete the device folder inside `storage/tmp_charon/<device>` as well (leaving only the `/ledgers` manifest and the remote S3 bucket). Run the query or click Export. SLiCER misses both Hot and Warm tiers, connects to AWS S3, downloads the encrypted envelopes, decrypts them on the fly, displays the events, and updates the **S3 Cost** badge.

8. REST API Reference Summary

Method & Endpoint	Payload / Params	Description
`POST /api/query`	`{ devices, fields, rawLuceneQuery, fromIsoDate, toIsoDate }`	Executes distributed search across tiers and streams back NDJSON chunks (`chunk` , `summary` , `error`).
`POST /api/export`	`{ device, blockName }`	Locates (Local → Charon → S3), decrypts, and exports a single block to `storage/exports/` .
`POST /api/export/selected`	`{ device, blockNames: [...] }`	Surgically exports only the list of blocks that matched the analyst's active query in the UI.
`GET /api/export/all/{device}`	Path param: `device`	Full forensic dump: consults local dirs, Charon, and the Central Ledger to export all historical blocks for a device.
`GET /api/health`	None	Health check endpoint returning `{"status": "SLiCER API is running"}` .